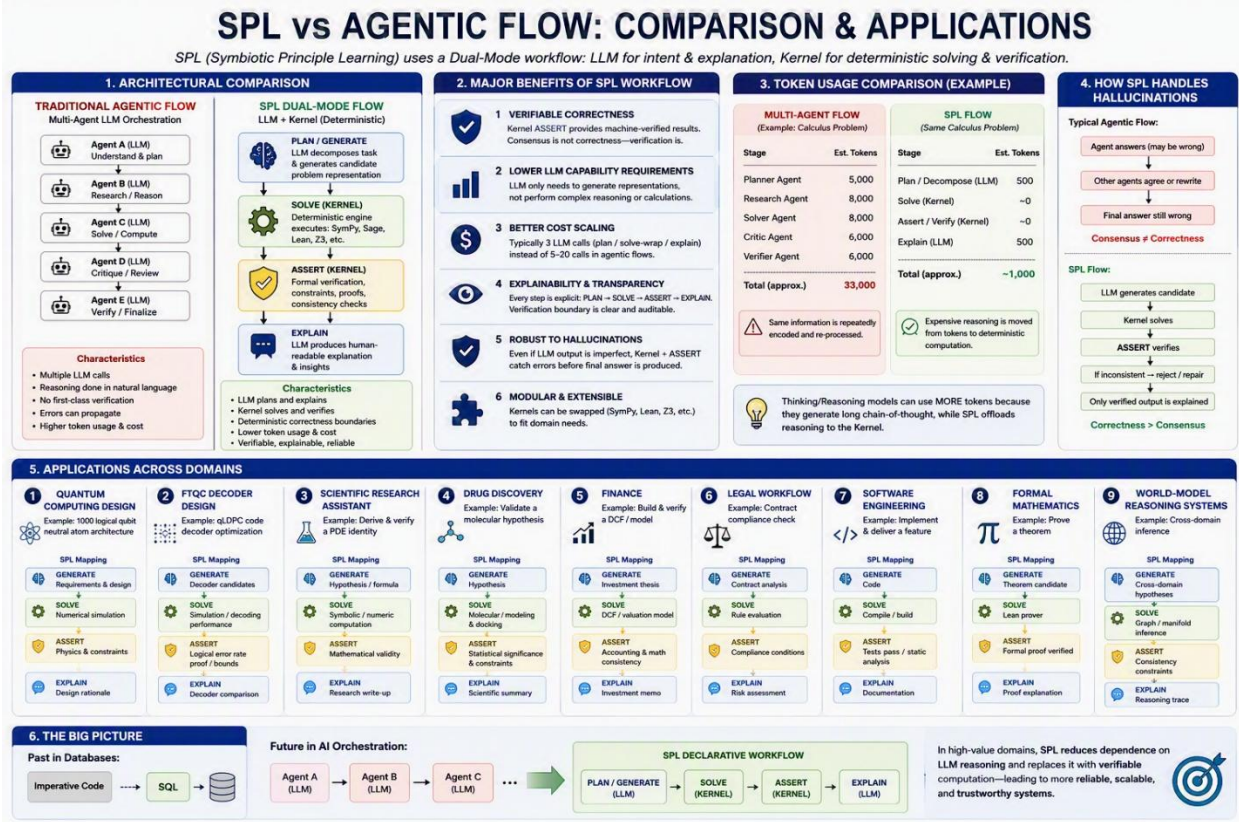


SPL vs Agentic Flow

Compiled by Zhengquan Tan, Aug. 6, 2026

based on the spl-dual-mode paper, analyze potential benefits using the SPL orchestrated workflow vs agents that calls different LLMs, list applications with examples

Summary in a diagram



Copilot said:

Based on the SPL paper, the biggest architectural difference is that **SPL is not "multi-agent orchestration" in the usual sense**. Instead, it is a **dual-mode orchestration system** that explicitly separates:

while most agent frameworks are:

Agent A (LLM)

↓

Agent B (LLM)

↓

Agent C (LLM)

↓

Agent D (LLM)

with no first-class deterministic reasoning layer.

SPL vs Multi-Agent LLM Architecture

Traditional Multi-Agent

Example:

Each step:

LLM → LLM → LLM → LLM

Advantages:

- flexible
- natural language
- easy to extend

Problems:

- hallucination propagation
- no formal verification
- expensive token usage
- inconsistent outputs

SPL Dual-Mode

Same task:

PLAN

|

GENERATE

|

▼

SOLVE

|

ASSERT

|

▼

EXPLAIN

becomes:

LLM

|



Kernel
(SymPy/Sage/Lean/Z3)

|



LLM
The expensive reasoning portion is replaced by deterministic execution.

Major Benefits of SPL Workflow

1. Verifiable Correctness

Agent workflows:

Agent A thinks result=42

Agent B agrees

Agent C rewords answer

Result:
still wrong
Consensus $\neq$ correctness.

SPL:

Generate derivation

|



ASSERT

|



Machine verified
Result:

Correctness guarantee
through SymPy, Sage, Lean, Z3, etc.

2. Lower LLM Capability Requirements

The paper's most interesting result:

Example:

Instead of:

LLM computes integral

SPL:

LLM

↓

Generate symbolic expression

SymPy

↓

Compute integral exactly

This dramatically reduces reasoning burden.

3. Better Cost Scaling

Multi-agent:

Often:

5-20 LLM calls
per request.

SPL:

1 planning call

1 solver call

1 explanation call

where solver cost may be nearly zero.

This can reduce token usage significantly.

4. Explainability

Typical agent systems:

Black-box chain

SPL:

PLAN

↓

SOLVE

↓

ASSERT

↓

EXPLAIN

Every transition is visible.

The verification boundary is explicit.

Application Areas

1. Quantum Computing Design

Your domain.

Traditional agents:

Requirements Agent

Architecture Agent

Cost Agent

Review Agent

may disagree.

SPL:

Example:

1000 logical qubit

neutral atom design

Verification layer:

checked numerically.

2. FTQC Decoder Design

Example:

Input:

qLDPC code

Workflow:

Result is substantially more trustworthy than agent consensus.

3. Scientific Research Assistant

Traditional:

Research Agent

Math Agent

Writing Agent

SPL:

Especially useful for:

- quantum mechanics
 - differential geometry
 - PDEs
 - control theory
-

4. Drug Discovery

Workflow:

SPL mapping:

GENERATE

Hypothesis

SOLVE

Molecular model

ASSERT

Statistical significance

GENERATE

Scientific summary

5. Finance

Agent system:

Analyst

Strategist

Reviewer

can still hallucinate.

SPL:

GENERATE

Investment thesis

SOLVE

DCF model

ASSERT

Accounting math

GENERATE

Investment memo

All calculations become deterministic.

6. Legal Workflow

GENERATE

Contract analysis

SOLVE

Rule evaluation

ASSERT

Compliance conditions

GENERATE

Risk assessment

Rule-based verification dramatically reduces errors.

7. Software Engineering

Current approach:

Coder Agent

Reviewer Agent

Tester Agent

SPL:

GENERATE code

SOLVE compile

ASSERT tests pass

GENERATE documentation

Very natural fit.

8. Formal Mathematics

This is arguably SPL's strongest use case.

Problem

↓

GENERATE theorem candidate

↓

LEAN PROVER

↓

ASSERT formal proof

↓

GENERATE explanation

Show more lines

This directly mirrors the paper's Lean verifier ladder.

9. World-Model Reasoning Systems

Using the topology/manifold ideas from your earlier work:

Physics

Math

AI

Language

Biology

can be represented as:

with:

GENERATE

Cross-domain hypotheses

SOLVE

Graph inference

ASSERT

Consistency constraints

GENERATE

Reasoning trace

A future "world model OS" is actually much closer to SPL than today's multi-agent chat loops.

Strategic Insight

The paper suggests a transition similar to:

1970s

Imperative Database Code

↓

SQL

and argues that AI orchestration may evolve from:

Agent A

↓

Agent B

↓

Agent C

into

Declarative Workflow

PLAN

↓

GENERATE

SOLVE

ASSERT

EXPLAIN

where LLMs become **probabilistic translators and planners**, while deterministic engines become the **source of truth**.

For high-value domains such as FTQC design, engineering optimization, scientific computing, theorem proving, finance, and software verification, the SPL architecture is likely more scalable than pure multi-agent systems because it reduces dependence on LLM reasoning and replaces it with verifiable computation whenever possible.

[Compare token usage in both models, How does SPL handle hallucinations? Explain the role of the kernel in SPL](#)

Copilot said:

The SPL paper reveals a surprisingly important result: **SPL often uses fewer tokens than multi-agent workflows because it offloads reasoning to deterministic kernels rather than repeatedly asking LLMs to think about the same problem.** The kernel is also the primary mechanism for hallucination containment and correctness verification.

1. Token Usage: SPL vs Multi-Agent LLM Systems

Traditional Multi-Agent Architecture

Typical workflow:

User Query

|

▼

Planner Agent

|

▼

Research Agent

|

▼

Solver Agent

|

▼

Critic Agent



Verifier Agent

Each stage may generate:

- chain-of-thought
- tool calls
- intermediate summaries
- critiques
- revisions

A simple engineering task can easily consume:

5–20 LLM calls

with the same information repeatedly re-encoded into prompts.

Example:

10k token context

Planner 5k tokens

Research 8k tokens

Solver 8k tokens

Critic 6k tokens

Reviewer 6k tokens

Total $\approx$ 33k tokens

before the final answer.

SPL Architecture

The SPL pattern is:

PLAN



GENERATE

SOLVE

ASSERT

EXPLAIN

Instead of:

LLM reasons

LLM checks

LLM rechecks
LLM validates
SPL performs:

LLM identifies problem

Kernel computes answer

Kernel verifies answer

LLM explains result
The expensive middle section becomes deterministic.

Example: Calculus Problem Multi-Agent

Agent 1:
recognize derivative

Agent 2:
compute derivative

Agent 3:
check derivative

Agent 4:
explain derivative
Potentially:

$2000 + 2000 + 2000 + 1500$

≈ 7500 tokens

SPL

GENERATE
"Differentiate $x^4 - 2x^2 + 1$ "

SOLVE
diff(...)

ASSERT
verification

GENERATE
explanation
Potentially:

500 token decomposition

0 token symbolic solve

0 token assertion

500 token explanation

≈ 1000 tokens

The mathematics move from token generation to symbolic execution.

Why Thinking Models Can Be Worse

The paper explicitly observed that some reasoning-heavy models:

qwen3

deepseek-r1

spent their token budget on extended thinking before producing the structured output required for SPL. The authors characterize this as a mismatch between the job required by SPL (translation) and the job performed by the model (full internal reasoning).

Conceptually:

Normal agent:

LLM solves problem

SPL:

LLM translates problem

Kernel solves problem

Thus SPL often lowers token consumption dramatically for symbolic tasks.

2. How SPL Handles Hallucinations

The paper treats hallucinations as a first-class workflow failure mode.

SPL introduces:

EXCEPTION

with conditions such as:

rather than treating hallucinations as ordinary text-generation errors.

Hallucination Defense Layer 1:

Structured Workflow

Instead of:

Prompt

↓

Answer

SPL creates multiple gates:

GENERATE

↓

EVALUATE

↓

SOLVE

↓

ASSERT

↓

RETURN

Each gate can reject invalid outputs.

Hallucination Defense Layer 2:

EVALUATE

Example:

GENERATE plan

EVALUATE plan quality

The generated content can be judged before downstream execution.

pass

fail

needs revision

This catches many planning errors.

Hallucination Defense Layer 3:

ASSERT

This is the strongest mechanism.

Example:

LLM:

"The derivative is x^2+4 "

ASSERT tests:

If false:

workflow fails verification

Rather than returning a hallucinated answer.

Hallucination Defense Layer 4:

Verifier Ladder

The SPL verifier hierarchy:

R1 SymPy

R2 SageMath

R3 Lean 4

provides increasing confidence.

Algebra

↓

Advanced Math

↓

Formal Proof

A claim that survives Lean verification is no longer merely plausible; it is machine-proved.

3. The Role of the Kernel in SPL

The kernel is the heart of the deterministic mode.

The paper divides SPL into:

System 1

Probabilistic

GENERATE

EVALUATE

and

System 2

Deterministic

SOLVE

ASSERT

The kernel implements System 2.

Function 1: Exact Computation

Instead of:

LLM estimates result

the kernel performs:

Algebra

Calculus

Linear Algebra

Geometry
Optimization
Logic
exactly.
Example:
executed by SymPy.

Function 2: Verification

The kernel checks whether claims are true.
This is analogous to:
rather than another conversational model.

Function 3: State Persistence

The kernel is persistent during execution.

Import libraries

Create variables

Store intermediate results

Reuse calculations
across steps.

Function 4: Formalization Boundary

One of the paper's most important concepts.

LLM Output

|

▼

@@var@@

|

▼

Kernel Input

The variable substitution layer translates:

Natural Language
into

Executable Symbolic Expressions

The kernel becomes the bridge from language to mathematics.

FTQC Example

For a 1000-logical-qubit FTQC architecture study, a traditional agent flow might be:

Architecture Agent
Cost Agent
Decoder Agent
Reviewer Agent
All reasoning remains probabilistic.
An SPL workflow could instead be:

GENERATE
FTQC architecture

SOLVE
Decoder throughput equations

SOLVE
Rack power model

SOLVE
CAPEX model

ASSERT
Timing constraints

ASSERT
Power constraints

GENERATE
Executive summary

The heavy numerical work moves into kernels:

Python
SymPy
NumPy
SageMath
Z3
while the LLM focuses on planning and communication.

Bottom Line		
Aspect	Multi-Agent LLMs	SPL Dual-Mode
Primary computation	LLM reasoning	LLM + deterministic kernels

Aspect	Multi-Agent LLMs	SPL Dual-Mode
Token consumption	High	Often much lower
Hallucination control	Critic/reviewer agents	ASSERT + verifier ladder
Correctness guarantee	No	Machine-verifiable
Cost scaling	More agents → more tokens	More computation → kernel
Best use	Open-ended exploration	Engineering, science, math, software, verification

The key insight of SPL is:

Use LLMs for:

Planning

Translation

Explanation

Use kernels for:

Computation

Verification

Proof

That separation dramatically reduces hallucination risk, lowers token usage for structured problems, and enables machine-verifiable workflows rather than consensus-based agent workflows.

Sources

How does SPL handle ambiguous prompts? Can SPL be integrated with external tools? Explain the role of the kernel in detail

How does SPL handle ambiguous prompts? Can SPL be integrated with external tools? Explain the role of the kernel in detail

Copilot said:

Based on the SPL paper, these three topics are closely related:

Ambiguous Prompt

↓

GENERATE / EVALUATE

↓

Kernel Formalization Boundary

↓

SOLVE / ASSERT

↓

Verified Result

A key insight is that SPL does **not try to eliminate ambiguity at the prompt level**. Instead, it manages ambiguity through workflow stages and pushes exactness into the deterministic kernel.

1. How SPL Handles Ambiguous Prompts

Traditional Agent Frameworks

Consider:

"Analyze the FTQC decoder architecture."

This is ambiguous:

- Performance analysis?
- Cost analysis?
- FPGA architecture?
- Decoder algorithm comparison?
- Real-time latency?

A multi-agent system might let several agents interpret the request differently:

Planner Agent

↓

Research Agent

↓

Reviewer Agent

Potential consequence:

Each agent may amplify the original misunderstanding.

SPL Approach

SPL separates:

Interpretation

from

Verification

The workflow might be:

GENERATE understanding

EVALUATE understanding

GENERATE plan

EVALUATE plan

SOLVE

ASSERT

The paper calls this:
rather than direct answer generation.

EVALUATE as Semantic Gate

The key mechanism is:

Example:

spl isn't fully supported. Syntax highlighting is based on .
GENERATE interpretation

EVALUATE interpretation

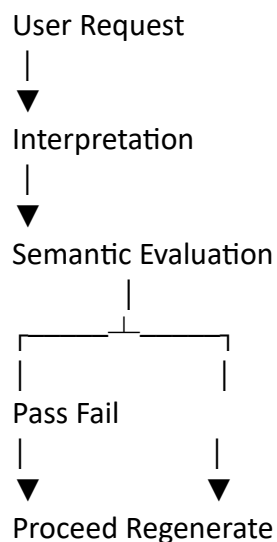
WHEN satisfactory
continue

ELSE
revise

This enables semantic quality checks before expensive downstream processing.

Ambiguity Resolution Loop

Conceptually:



Unlike agent-to-agent chains, ambiguity can be trapped early.

2. Can SPL Integrate External Tools?

The answer is essentially **yes by design**.

One of the strongest claims of the paper is that any Python-callable capability can be integrated into the deterministic side.

Tool Integration Model

The paper exposes:
meaning:

LLM

|



Tool API

|



Workflow Variable

The tool becomes a first-class workflow component.

Examples Mentioned in the Paper

The paper explicitly discusses integration with:
and extension to:

NetworkX

Z3

mypy

SciPy

constraint solvers

statistical engines

through the same interface.

Engineering Example

For FTQC analysis:

GENERATE

Architecture proposal

CALL

Cost model

CALL

Resource estimator

CALL
Decoder simulator

ASSERT
Timing constraints
where:

Cost model
Decoder simulator
Resource estimator
may all be external tools.

Scientific Workflow Example

GENERATE hypothesis

CALL molecular model

CALL simulation

ASSERT statistical validity

GENERATE report
This resembles a scientific workflow engine more than an agent framework.

Why This Matters

Most agent frameworks connect tools through:
Python
Agent -> Tool

inside custom code.
SPL elevates tool integration into the language:

GENERATE
SOLVE
ASSERT
CALL
making the workflow specification independent of the implementation.

3. The Kernel in SPL: Detailed Architecture

This is arguably the most important component of SPL.
The paper repeatedly divides the system into:

System 1
Probabilistic

and

System 2
Deterministic
with the kernel implementing System 2.

Kernel as the Deterministic Engine

Conceptually:

LLM=Hypothesis Generator

Kernel=Truth Maintenance System
The kernel performs exact computation.

Layer 1:

Execution Engine

The kernel executes:
rather than generating text.

Example:

Kernel result:

$4 * x^3$

No token generation required.

Layer 2:

Verification Engine

The kernel validates claims.

Example:

spl isn't fully supported. Syntax highlighting is based on .

ASSERT

simplify(candidate - truth)==0

A passing assertion means:

Machine verified

rather than:

This is the foundation of SPL's hallucination control.

Layer 3:

Formalization Boundary

One of the most important ideas in the paper.

Natural Language



GENERATE



@@var@@



Kernel

The paper explicitly identifies:

@@variable@@

substitution as the mode boundary between:

System 1

(probabilistic)

and

System 2

(deterministic)

The runtime automatically converts workflow variables into executable expressions.

Layer 4:

Persistent State

The kernel persists through the workflow.

Define variables

Import libraries

Store results

Reuse intermediate computations
across multiple SOLVE and ASSERT steps.

Example:

can occur inside one kernel session.

Layer 5:

Verifier Ladder

The kernel supports multiple verification tiers.

R1 SymPy



R2 SageMath



R3 Lean 4

Each rung increases confidence.

SymPy

Best for:

Algebra

Calculus

Matrices

Basic symbolic math

SageMath

Best for:

Number theory

Geometry

Advanced symbolic computation

ODEs

Transforms

Lean 4

Best for:

A Lean-verified result becomes a proof artifact rather than merely a computation.

FTQC Example

For a 1000 logical-qubit neutral-atom FTQC architecture:

User Request



GENERATE

Subsystem architecture



GENERATE

Resource equations



SOLVE
Decoder throughput



SOLVE
Laser counts



SOLVE
Power budget



ASSERT
Timing margin > 20%



ASSERT
Rack power < target



GENERATE
Executive summary

The LLM contributes:

Planning
Interpretation
Narration
while the kernel contributes:

Executive Summary

Ambiguity Handling=GENERATE + EVALUATE feedback loops

External Tool Integration=CREATE TOOL_API + CALL + SOLVE

Kernel=Deterministic System-2 Engine

Functions:

- Exact computation
- Verification
- State management
- Tool execution
- Formal proof
- Constraint checking

Verifier Ladder:

SymPy → SageMath → Lean

LLMs provide:

Planning

Interpretation

Explanation

Kernel provides:

Truth

Correctness

Reproducibility

Show more lines

The deepest architectural contribution of SPL is that it treats the kernel not as a tool attached to an LLM, but as a **co-equal computational mode**. The LLM explores possibilities; the kernel certifies reality.

Sources

How does SPL handle real-time data? Can SPL be used for legal reasoning? Explain the role of @@var@@ in SPL

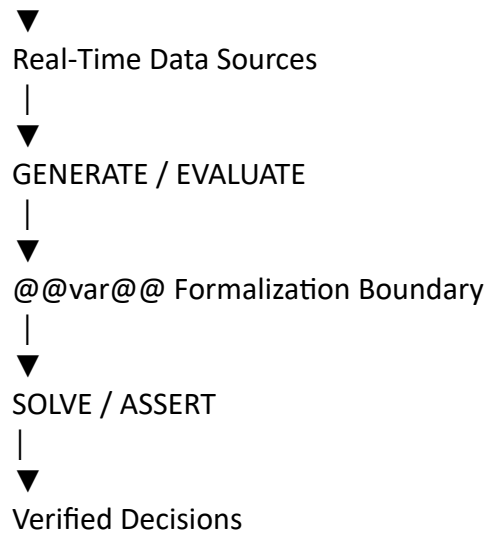
How does SPL handle real-time data? Can SPL be used for legal reasoning? Explain the role of @@var@@ in SPL

Copilot said:

According to the SPL paper, these three questions converge on a central theme:

External World

|



The SPL architecture is designed to bridge probabilistic reasoning (LLMs) and deterministic computation (kernels/tools). Real-time data, legal reasoning, and @@var@@ all sit at that boundary.

1. How SPL Handles Real-Time Data

Native Approach

SPL itself is not a database or streaming engine.

Instead, it treats real-time information as external inputs that become part of the workflow state. The paper's architecture allows:

RAG
Memory
Tool APIs
External Services
Cloud APIs
to feed variables into the workflow before subsequent reasoning steps.
Conceptually:

Market Feed
Weather Feed
Enterprise Data
Sensor Data
Research Papers



Tool API





@current_data



GENERATE

SOLVE

ASSERT

Example: FTQC Operations

Imagine a future FTQC operations center.

Real-time inputs:

Decoder throughput

GPU utilization

Atom loss telemetry

Laser stability

Rack power

Workflow:

spl isn't fully supported. Syntax highlighting is based on .

CALL telemetry_feed()

INTO @telemetry

GENERATE anomaly_assessment(@telemetry)

SOLVE throughput_model(@@telemetry@@)

ASSERT latency < threshold

The LLM interprets data.

The kernel verifies constraints.

This is precisely the type of deterministic/probabilistic composition SPL was designed for.

Event-Driven Workflows

The paper's architecture naturally supports:

Data arrives



Workflow triggered



Tool retrieves state

↓
GENERATE

↓
SOLVE

↓
ASSERT

↓
Action

Examples:

- financial monitoring
- cybersecurity
- industrial control
- scientific experiments
- cloud operations

Because tool APIs are first-class citizens, real-time data is simply another workflow input.

2. Can SPL Be Used for Legal Reasoning?

Short answer:

Yes

but with important distinctions.

The SPL paper explicitly argues that the verifier ladder can be extended beyond mathematics to any Python-callable verification mechanism.

Traditional Legal LLM Workflow

Typical workflow:

Contract Agent

↓

Compliance Agent

↓

Risk Agent

↓

Reviewer Agent

Problems:

SPL Legal Workflow

SPL separates:

Interpretation

from

Verification

Example:

The LLM performs:

Language understanding
Contract interpretation
Explanation
while deterministic systems perform:

Rule checking
Clause matching
Regulatory validation
Constraint enforcement

Compliance Example

Suppose the workflow evaluates:
Pipeline:

Contract
↓
GENERATE
Extract obligations
↓
SOLVE
Compare against policy rules
↓
ASSERT
All mandatory clauses exist
↓
GENERATE
Compliance report

This dramatically reduces hallucination risk compared with pure LLM reasoning.

Regulatory Workflows

The SPL cookbook already includes examples such as:
which follow the same pattern:

Interpret
Compute
Verify
Report
rather than only:

Interpret
Interpret

Interpret

3. The Role of @@var@@ in SPL

This is arguably the most important concept in the paper.

The authors explicitly describe @@var@@ as the **formalization boundary** between System 1 and System 2.

System 1 vs System 2

In SPL:

System 1
(Probabilistic)

GENERATE
EVALUATE

System 2
(Deterministic)

SOLVE
ASSERT
These two worlds must exchange information.
@@var@@ is how that happens.

Example

Suppose the LLM creates:

@expression = "x**4 - 2*x**2 + 1"

Then:

spl isn't fully supported. Syntax highlighting is based on .

SOLVE @derivative :=

"diff(@@expression@@, x)"

The runtime replaces:

@@expression@@

with:

$x^{**4} - 2*x^{**2} + 1$

before sending the command to the kernel.

Conceptual Meaning

Without @@var@@:

LLM world
and

Kernel world
are disconnected.

With @@var@@:

Natural Language

↓

Workflow Variable

↓

The variable crosses the mode boundary.

Why It Is So Important

The paper emphasizes that users do not manually marshal data between:

LLM

⇔

Kernel

Instead:

handles the transfer automatically.

Example:

spl isn't fully supported. Syntax highlighting is based on .

```
GENERATE decomposition(@problem)
```

```
INTO @steps
```

```
SOLVE @result :=
```

```
"solve_chain(@@steps@@)"
```

```
ASSERT verify(@@result@@)
```

The workflow author never writes serialization glue code.

@@var@@ as a Type Conversion Layer

From an architectural perspective:

@var

represents:

Workflow State
while

@@var@@
represents:

Executable Deterministic Input
You can think of it as:

Language Space
↓
Formalization
↓
Symbolic Space

World-Model Interpretation

Using your manifold/world-model framework:

Physics
Math
AI
Language
Biology
the flow becomes:

Semantic Manifold
↓
GENERATE
↓
@concept
↓
@@concept@@
↓
Kernel Computation
↓
ASSERT
↓
Verified Knowledge
Show more lines

In that sense, @@var@@ is the mathematical "projection operator" that converts a probabilistic semantic representation into a deterministic symbolic representation.

Executive Summary

Real-Time Data

- Handled through tool APIs, RAG, memory, and external services
- Fed into workflow variables
- Can participate in GENERATE, SOLVE, and ASSERT

Legal Reasoning

- Strong fit for compliance, auditing, and rule-based validation
- LLM interprets language
- Deterministic tools verify regulations and constraints

@@var@@

- Most important SPL integration mechanism
- Formalization boundary between LLMs and kernels
- Converts workflow state into executable symbolic expressions
- Eliminates manual glue code
- Enables deterministic verification and computation

Show more lines

The deepest contribution of @@var@@ is that it acts as the bridge between **meaning** and **mechanism**: LLMs generate semantic intent, @@var@@ formalizes that intent, and the kernel turns it into verifiable computation.

Sources

How does SPL handle streaming data? Can SPL be used for real-time decision-making? Explain how @@var@@ supports formal verification

How does SPL handle streaming data? Can SPL be used for real-time decision-making? Explain how @@var@@ supports formal verification

Copilot said:

Based on the SPL paper, these three concepts form a continuous pipeline:

Streaming Data

↓

Workflow State (@var)

↓

Formalization Boundary (@@var@@)

↓

Kernel Verification

↓

Real-Time Decisions

The key insight is that SPL was designed around **stateful workflows**, where probabilistic reasoning (LLMs) and deterministic verification (kernels) can repeatedly interact as new information arrives.

1. How SPL Handles Streaming Data

The paper primarily discusses workflow execution rather than streaming systems, but its architecture naturally supports streaming through:

- shared workflow state (@variables)
- external tool APIs
- memory and retrieval primitives
- iterative loops (WHILE)
- continuous evaluation (EVALUATE)

Conceptual Model

Telemetry Stream

↓

Market Feed

↓

Sensor Data

↓

CALL Tool API

↓

@current_state

↓

GENERATE

↓

EVALUATE

↓

SOLVE

↓

ASSERT

Rather than storing all data inside the LLM context, SPL keeps incoming information in workflow variables and deterministic tools.

Example: FTQC Operations Center

For a 1000-logical-qubit FTQC system:

Streaming inputs:

Syndrome rate

Atom loss rate

GPU utilization

FPGA queue depth

Laser drift

Vacuum pressure

Workflow:

CALL telemetry_feed()

↓

@telemetry
↓
GENERATE anomaly report
↓
SOLVE decoder throughput
↓
ASSERT timing margin

The LLM interprets the evolving system state while the kernel validates quantitative constraints.

WHY SPL Is Better Than Agent Loops

Many agent frameworks repeatedly send the entire state back through LLMs:

Data
↓
Agent A
↓
Agent B
↓
Agent C
SPL instead separates:

Interpretation
from
Verification
allowing large portions of the state to remain in deterministic systems.

2. Can SPL Be Used for Real-Time Decision-Making?

Yes. In fact, the PLAN → SOLVE → ASSERT → ACT pattern is one of the most natural applications of SPL.

Generic Decision Pipeline

Incoming Event
↓
GENERATE
Situation Assessment
↓
SOLVE
Exact Calculations
↓
ASSERT

Safety Constraints

↓

Decision

The important difference from a pure agent architecture is that decisions do not rely solely on LLM confidence.

They can be gated by machine-verifiable rules.

Financial Example

Market Data

↓

GENERATE

Risk Analysis

↓

SOLVE

Portfolio Exposure

↓

ASSERT

Risk < Threshold

↓

Trade / No Trade

Show more lines

Instead of:

Agent thinks trade is good.

SPL enables:

Agent proposes.

Kernel verifies.

Autonomous Infrastructure

Example:

Cloud utilization > 85%

↓

GENERATE

Scaling recommendation

↓

SOLVE

Expected capacity

↓

ASSERT

SLA maintained

↓

Deploy

Show more lines

This is much closer to industrial control workflows than traditional chat-based agents.

FTQC Example

Imagine real-time decoder management:

Syndrome backlog spike

↓

GENERATE

Likely root cause

↓

SOLVE

Predicted latency

↓

ASSERT

Latency < QEC budget

↓

Schedule additional GPUs

Every decision can be constrained by deterministic timing and resource models. That is exactly the sort of workflow SPL was designed to support.

3. How @@var@@ Supports Formal Verification

This is arguably the most important concept in SPL.

The paper explicitly identifies:

@@var@@

as the **formalization boundary** between:

System 1

(probabilistic)

and

System 2

(deterministic)

Formal verification requires:

A conversion must occur.

What @@var@@ Does

Suppose:

Inside a SOLVE:

diff(@@expression@, x)
the runtime substitutes:

$x^4 - 2x^2 + 1$
before sending it to the kernel.
Conceptually:

Language
↓
@expression
↓
@@expression@@
↓
Executable Symbol
The semantic result becomes a formal object.

Why Formal Verification Depends on @@var@@

Without @@var@@:

LLM answer

=

The kernel cannot verify it.

With @@var@@:

The deterministic engine now has something it can compute and prove.

Mathematical Example

Step 1: Generate

@derivative =
"4*x**3"

Step 2: Formalize

@@derivative@@

Step 3: Verify

ASSERT

simplify(
@@derivative@@
-
diff(x**4,x)

) == 0

The kernel evaluates:

Result:

Machine verified

rather than merely plausible.

Lean Formal Proof Example

At the highest verifier rung:

GENERATE

theorem statement

↓

@@theorem@@

↓

LEAN KERNEL

↓

ASSERT proof_valid

The same @@var@@ mechanism transports information from LLM-generated mathematics into a formal proof assistant.

Deep Architectural View

The SPL paper describes a shared variable space:
and a formalization boundary:

@@var@@

Together they create:

SYSTEM 1

GENERATE

EVALUATE

↓

@problem

@plan

@claim

↓

FORMALIZATION

@@var@@

↓

SYSTEM 2

↓

SOLVE

ASSERT

↓
Verified Result
Show more lines

Key Insight

The deepest role of @@var@@ is not variable substitution.
It acts as a **semantic-to-symbolic compiler**:

Meaning
↓
LLM Representation
↓
@var
↓
@@var@@
↓
Formal Object
↓
Kernel Verification
``

This is why SPL can support:

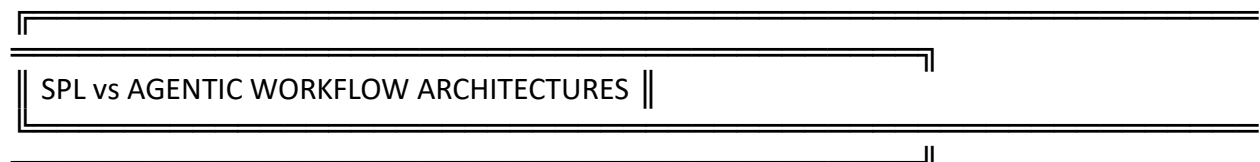
- real-time decision systems
- regulatory compliance engines
- scientific workflows
- theorem proving
- FTQC resource management

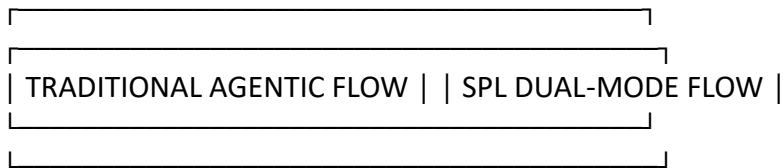
while maintaining deterministic verification. The LLM provides hypotheses and interpretations; @@var@@ converts them into formal objects; the kernel establishes whether they are actually true.

make a visual diagram to summarize the difference between SPL and agentic flow with example of applications

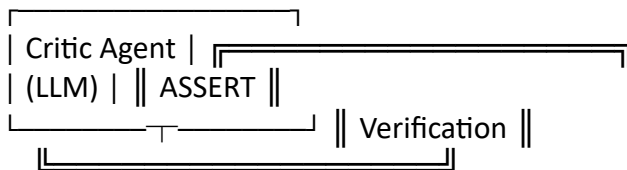
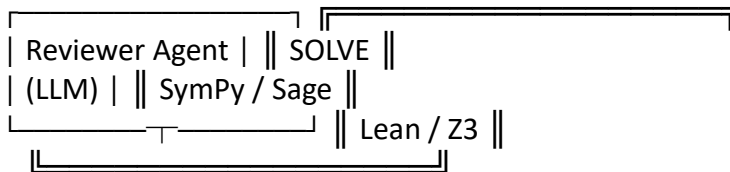
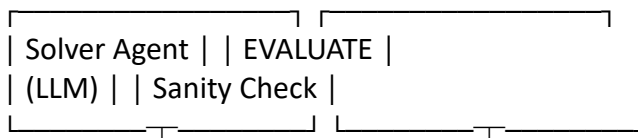
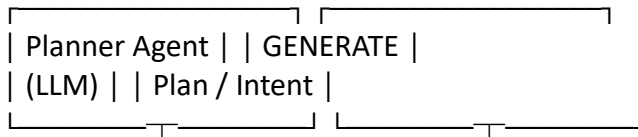
Copilot said:

SPL vs Traditional Agentic Flow

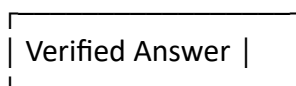
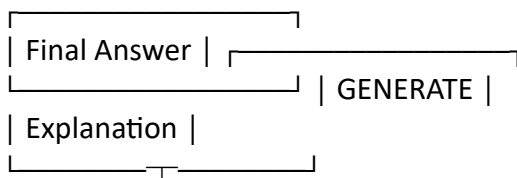




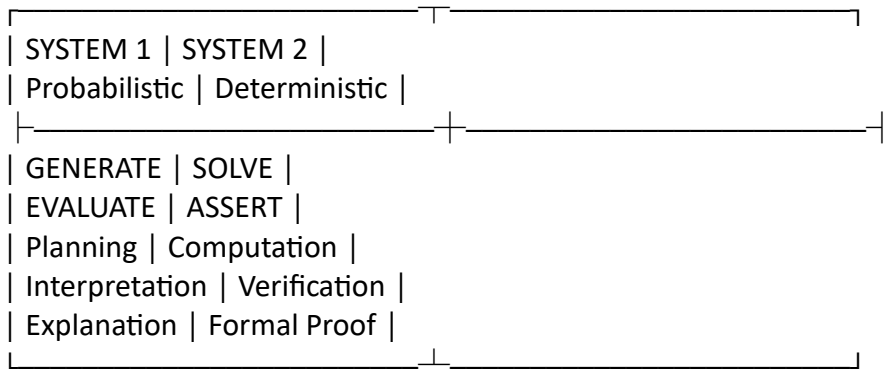
User Query User Query



PASS?

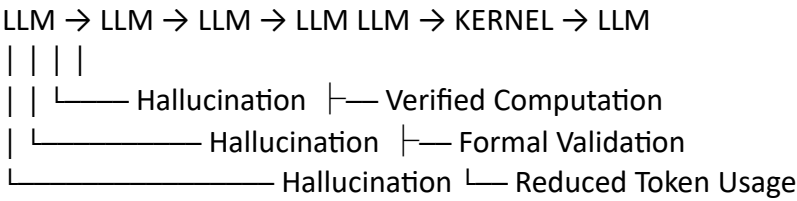


SYSTEM RESPONSIBILITY SPLIT



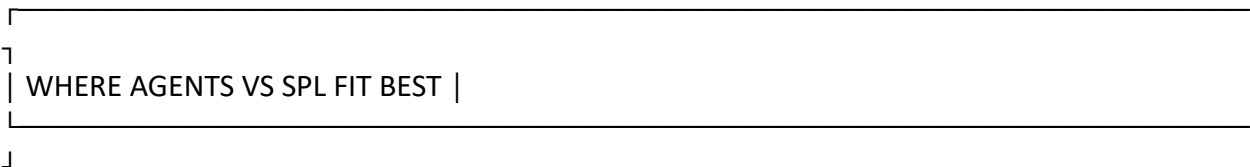
TOKEN & HALLUCINATION FLOW

AGENTIC FLOW SPL FLOW



High Token Cost Lower Token Cost
Consensus-Based Verification-Based
Plausible Answers Certified Answers
,

Application Mapping



OPEN-ENDED TASKS VERIFIED TASKS
(Agentic Strength) (SPL Strength)

Marketing Campaign	Mathematical Proof
Creative Writing	Scientific Research
Brainstorming	Software Testing
Product Ideation	Financial Modeling
Strategy Discussion	Engineering Design
	Legal Compliance
FTQC Architecture	

Examples:

FTQC Design

GENERATE → Architecture Proposal
SOLVE → Decoder Throughput Model
ASSERT → Timing Budget Validation
GENERATE → Executive Report

Legal Compliance

GENERATE → Contract Interpretation
SOLVE → Rule Evaluation
ASSERT → Regulatory Compliance
GENERATE → Risk Assessment

Scientific Computing

GENERATE → Solution Strategy
SOLVE → Symbolic Computation
ASSERT → Mathematical Verification
GENERATE → Human Explanation

Software Engineering

GENERATE → Code

SOLVE → Compile/Test
ASSERT → Unit Tests Pass
GENERATE → Documentation

Real-Time FTQC Operations

Telemetry
↓
GENERATE → Anomaly Hypothesis
↓
SOLVE → Resource Models
↓
ASSERT → QEC Timing Constraints
↓
Action Decision
Show more lines
Executive Takeaway

Agentic Flow:
"Multiple LLMs try to agree."

SPL:
"LLM proposes, kernel proves."

Agentic Flow optimizes:
Creativity, exploration, collaboration

SPL optimizes:
Correctness, verification, reproducibility

Best Future Architecture:
Agentic Planning
+
SPL Verification
=
Trustworthy AI Systems

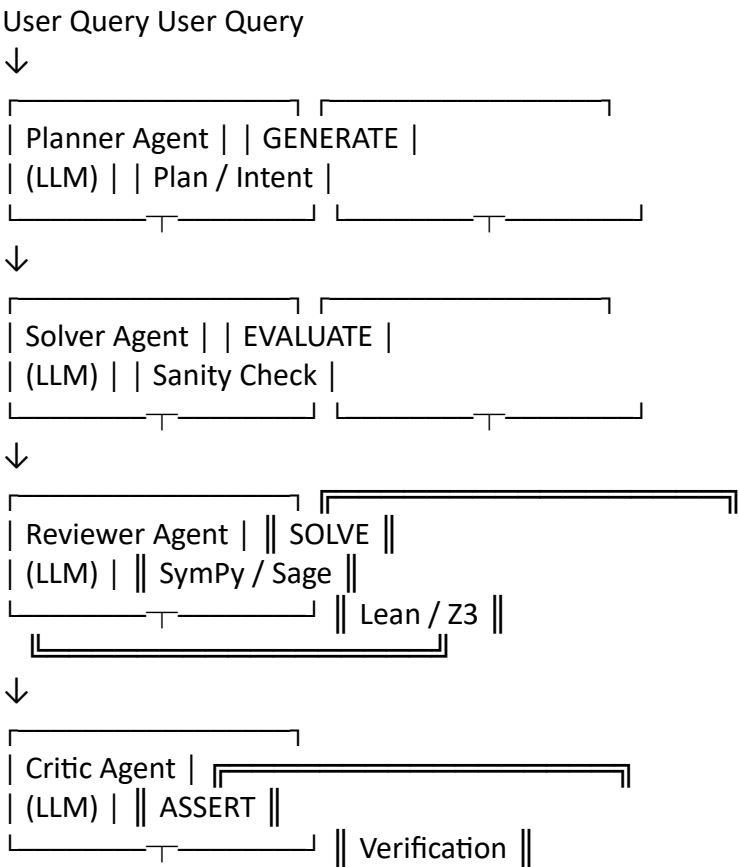
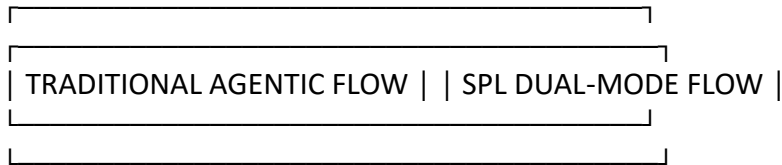
The SPL paper's core innovation is that it elevates **deterministic computation (SOLVE/ASSERT)** to the same level as **LLM reasoning (GENERATE/EVALUATE)**, creating a true **System-1 + System-2 architecture** rather than an LLM-only agent network.

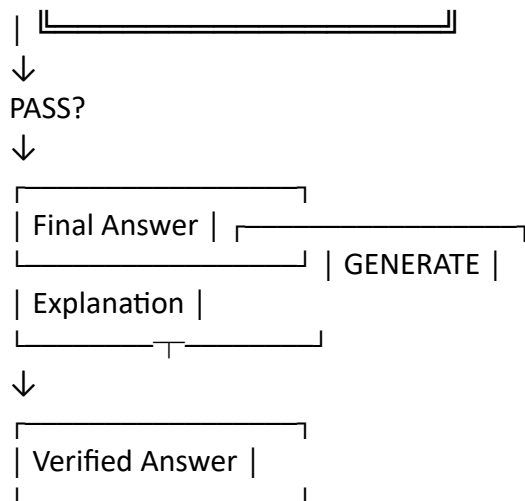
Sources

Explain the role of @@var@@ in formal verificationHow does SPL reduce hallucinations?What are the limitations of SPL?

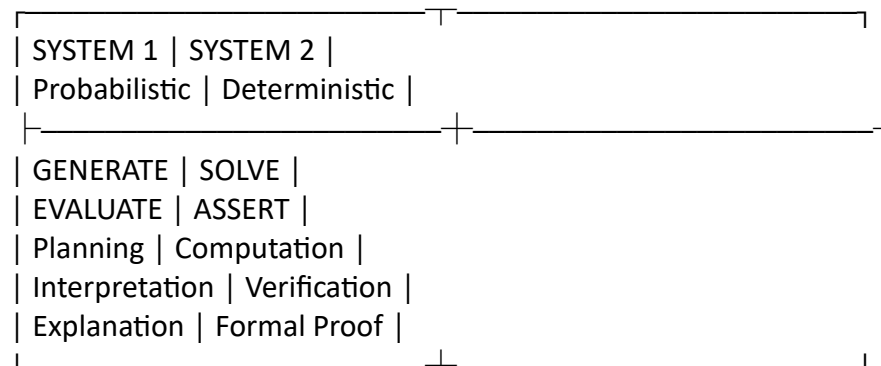
make a visual diagram to summarize the difference between SPL and agentic flow with example of applications

Copilot said:
SPL vs Traditional Agentic Flow



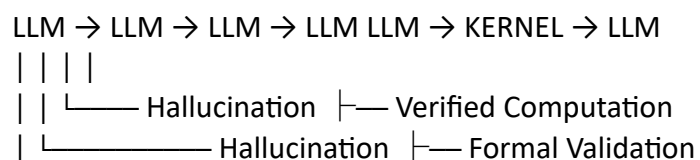


SYSTEM RESPONSIBILITY SPLIT



TOKEN & HALLUCINATION FLOW

AGENTIC FLOW SPL FLOW



_____ Hallucination _____ Reduced Token Usage

High Token Cost Lower Token Cost
Consensus-Based Verification-Based
Plausible Answers Certified Answers
,

Application Mapping

┌
└
| WHERE AGENTS VS SPL FIT BEST |
└
└

OPEN-ENDED TASKS VERIFIED TASKS
(Agentic Strength) (SPL Strength)

┌ Marketing Campaign | | Mathematical Proof |
┌ Creative Writing | | Scientific Research |
┌ Brainstorming | | Software Testing |
┌ Product Ideation | | Financial Modeling |
┌ Strategy Discussion | | Engineering Design |
└ | Legal Compliance |
┌ FTQC Architecture |
└

Examples:

FTQC Design

GENERATE → Architecture Proposal
SOLVE → Decoder Throughput Model
ASSERT → Timing Budget Validation
GENERATE → Executive Report

Legal Compliance

GENERATE → Contract Interpretation
SOLVE → Rule Evaluation

ASSERT → Regulatory Compliance
GENERATE → Risk Assessment

Scientific Computing

GENERATE → Solution Strategy
SOLVE → Symbolic Computation
ASSERT → Mathematical Verification
GENERATE → Human Explanation

Software Engineering

GENERATE → Code
SOLVE → Compile/Test
ASSERT → Unit Tests Pass
GENERATE → Documentation

Real-Time FTQC Operations

Telemetry
↓
GENERATE → Anomaly Hypothesis
▼
SOLVE → Resource Models
▼
ASSERT → QEC Timing Constraints
▼
Action Decision

Executive Takeaway

Agentic Flow:
"Multiple LLMs try to agree."

SPL:
"LLM proposes, kernel proves."

Agentic Flow optimizes:
Creativity, exploration, collaboration

SPL optimizes:

Correctness, verification, reproducibility

Best Future Architecture:

Agentic Planning

+

SPL Verification

=

Trustworthy AI Systems

The SPL paper's core innovation is that it elevates **deterministic computation (SOLVE/ASSERT)** to the same level as **LLM reasoning (GENERATE/EVALUATE)**, creating a true **System-1 + System-2 architecture** rather than an LLM-only agent network.

Sources

You said:

make a nice visual diagram per description

| WHERE AGENTS VS SPL FIT BEST |

└ OPEN-ENDED TASKS VERIFIED TASKS (Agentic Strength) (SPL Strength)

| Marketing Campaign | | Mathematical Proof |

| Creative Writing | | Scientific Research |

| Brainstorming | | Software Testing |

| Product Ideation | | Financial Modeling |

| Strategy Discussion | | Engineering Design |

| Legal Compliance | | FTQC Architecture |

Examples:

FTQC Design ——— GENERATE → Architecture Proposal SOLVE → Decoder Throughput
Model ASSERT → Timing Budget Validation GENERATE → Executive Report

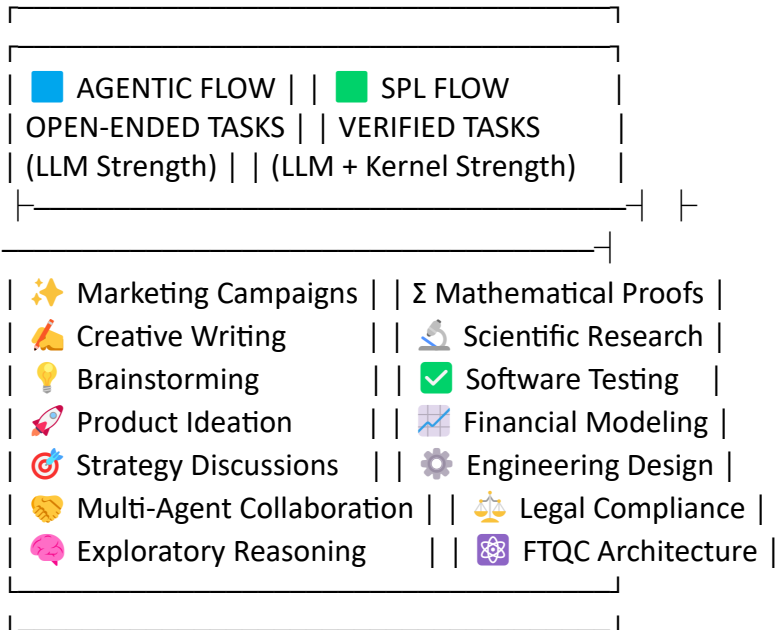
Legal Compliance ——— GENERATE → Contract Interpretation SOLVE → Rule
Evaluation ASSERT → Regulatory Compliance GENERATE → Risk Assessment

Scientific Computing ——— GENERATE → Solution Strategy SOLVE →
Symbolic Computation ASSERT → Mathematical Verification GENERATE → Human Explanation

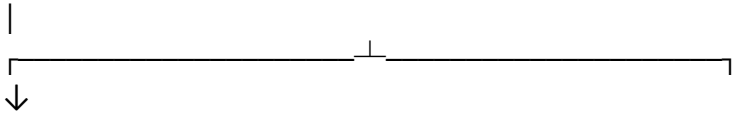
Software Engineering ——— GENERATE → Code SOLVE → Compile/Test
ASSERT → Unit Tests Pass GENERATE → Documentation

Real-Time FTQC Operations ——— Telemetry → GENERATE → Anomaly
Hypothesis → SOLVE → Resource Models → ASSERT → QEC Timing Constraints → Action
Decision

Copilot said:



Hybrid Enterprise AI



Agentic Creativity SPL Verification Engine

"What could work?" "Can we prove it works?"

EXAMPLE 1 : FTQC ARCHITECTURE DESIGN

GENERATE |
Architecture |
Proposal |

↓

SOLVE |
Decoder Throughput |
Resource Models |

↓

ASSERT |
Timing Budget |
Error Budget |
Power Limits |

↓

GENERATE |
Executive Report |

EXAMPLE 2 : LEGAL COMPLIANCE

Contract

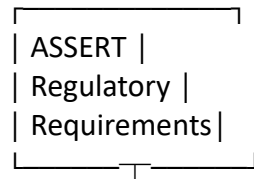
↓

GENERATE |
Interpret |
Clauses |

↓

SOLVE |
Rule Engine |
Compliance |

↓



↓



EXAMPLE 3 : SCIENTIFIC COMPUTING

Research Problem

↓

GENERATE

Strategy / Plan

↓

SOLVE

Symbolic Math

Simulation

Optimization

↓

ASSERT

Mathematical

Verification

↓

GENERATE

Human Explanation

Paper Draft

Report

EXAMPLE 4 : SOFTWARE ENGINEERING

Requirements

↓

GENERATE

Source Code

↓

SOLVE

Compile

Execute

↓

ASSERT

Unit Tests

Integration Tests

↓

GENERATE

Documentation

Release Notes

EXAMPLE 5 : REAL-TIME FTQC OPERATIONS

Telemetry Stream

Syndrome Rate

Atom Loss

FPGA Utilization

GPU Load

Laser Drift

↓

| GENERATE |

| Anomaly Analysis |

↓

| SOLVE |

| Throughput Model |
| Resource Model |

```
graph TD; A["| Throughput Model |  
| Resource Model |"] --> B["| ASSERT |  
| QEC Timing |  
| Safety Limits |"]; B --> C["| ACTION DECISION |  
| Allocate GPUs |  
| Repair Patch |  
| Schedule Qubits |"];
```

↓

| ASSERT |
| QEC Timing |
| Safety Limits |

↓

| ACTION DECISION |
| Allocate GPUs |
| Repair Patch |
| Schedule Qubits |

KEY DIFFERENCE

AGENTIC FLOW SPL FLOW

LLM → LLM → LLM → LLM GENERATE → SOLVE → ASSERT → GENERATE

Consensus-Based Verification-Based

"Agents agree" "Kernel proves"

Exploration Certification

Creativity Correctness

Flexible Auditable

Higher Token Usage Lower Token Usage

Hallucination Possible Assertion-Gated

Best for Ideation Best for Engineering / Science / FTQC

Best Future Architecture

Agentic Exploration

+

SPL Verification

=

Trustworthy AI Systems

Add icons to enhance visual clarity
Explain the role of the kernel
How does SPL reduce hallucinations?